UNIVERSIDAD NACIONAL SAN AGUSTÍN DE AREQUIPA
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE CIENCIA DE LA COMPUTACIÓN



Laboratorio 5: Pruebas Unitarias y Dobles de Pruebas

Docentes : Muñoz Salas, Gina
Sarmiento Calisaya, Edgar

Curso : Ingeniería de Software II

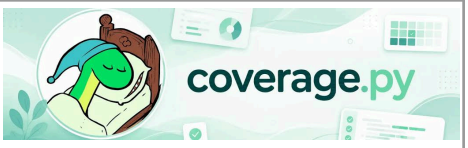
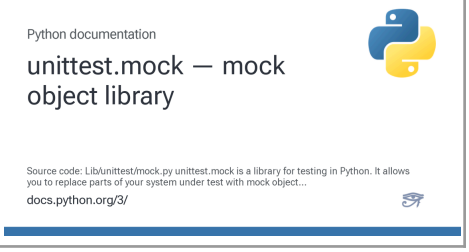
Alumnos : Ccaso Idme, Moises Robert
Huaylla Guzmán, Roid Niro
Quispe Torrez, Job Lorenzo
Rivera Olivera, Omar Joaquín
Sucari Ccansaya, Edú André
Torres Chávez, Jhair Alejandro

Arequipa - Perú

2026

1. Herramientas y tecnologías elegidas para pruebas

Para la implementación de las pruebas automatizadas se seleccionaron herramientas del ecosistema Python que permiten cubrir las principales necesidades del proceso de testing. Estas tecnologías facilitan la creación y ejecución de pruebas unitarias, el aislamiento de dependencias mediante *mocking* y la medición de la cobertura del código, contribuyendo a mejorar la calidad y confiabilidad del software.

Especificación	Tecnología / Herramienta	Propósito principal	Imagen
Lenguaje de programación	***Python***	Lenguaje utilizado para el desarrollo de la aplicación y la implementación de las pruebas.	
Framework xUnit	***unittest***	Creación, organización y ejecución de pruebas unitarias.	
Herramienta de cobertura	***Coverage.py***	Medición del porcentaje de código ejecutado durante las pruebas.	
Framework de mocking	***unittest.mock***	Creación de objetos simulados (*mocks*) para aislar componentes y dependencias durante las pruebas.	

2. Separación de código de implementación y pruebas

- La separación del código de implementación y las pruebas permite **mantener una estructura organizada y facilitar el mantenimiento del proyecto**, evitando mezclar la lógica de negocio con el código destinado a verificar su correcto funcionamiento. Además, permite ejecutar, modificar y gestionar las pruebas de forma independiente sin afectar directamente al código de producción.

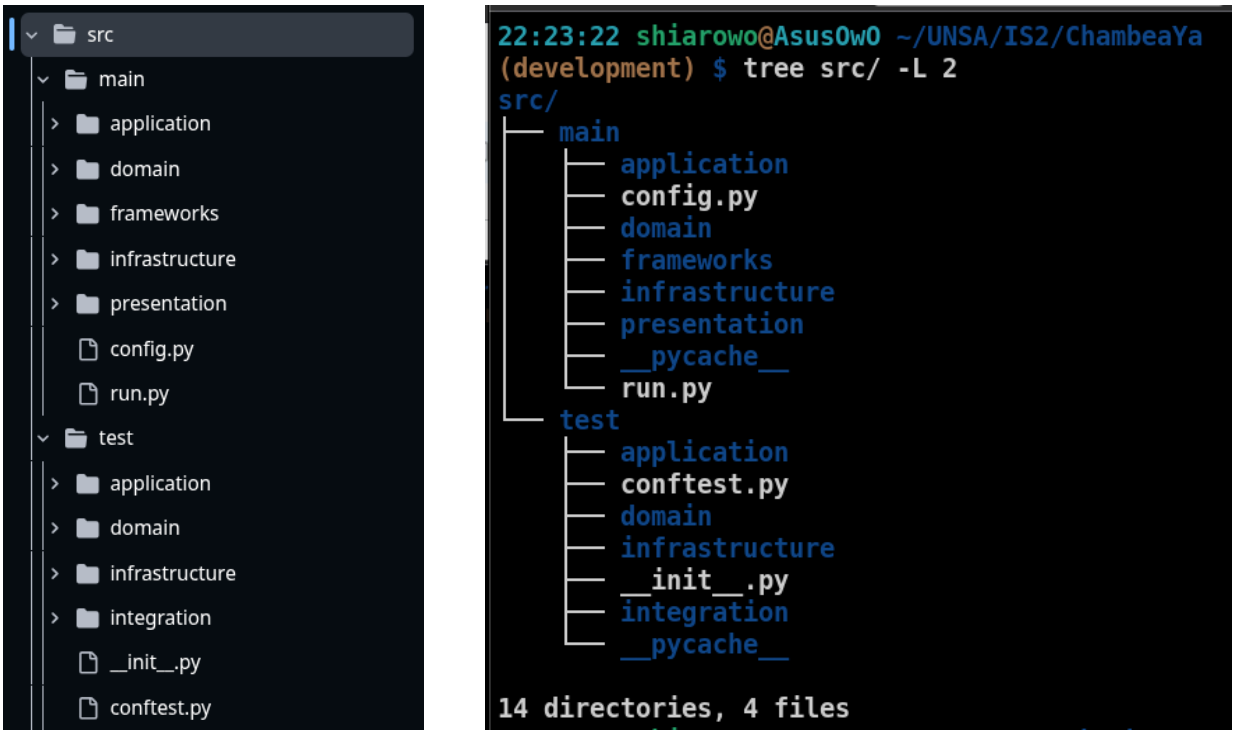


Figura 1: Estructura de la carpeta src/ dentro del repositorio con clara división de las carpetas 'main' (implementación) y 'test' (pruebas)

3. Creación de los issues y GitHub Project (Kanban)

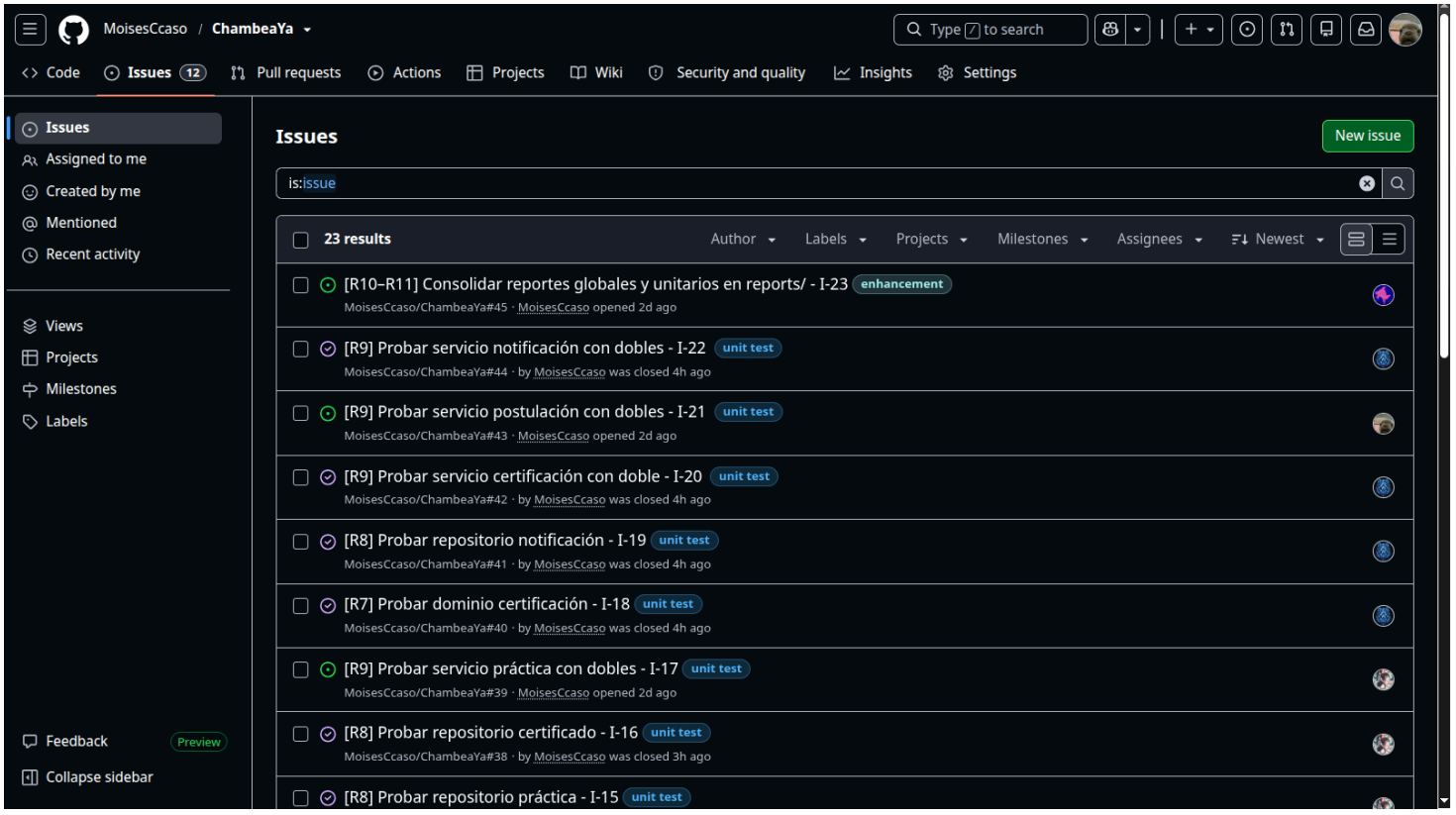


Figura 2: Registro de los Issues dentro del repositorio GitHub

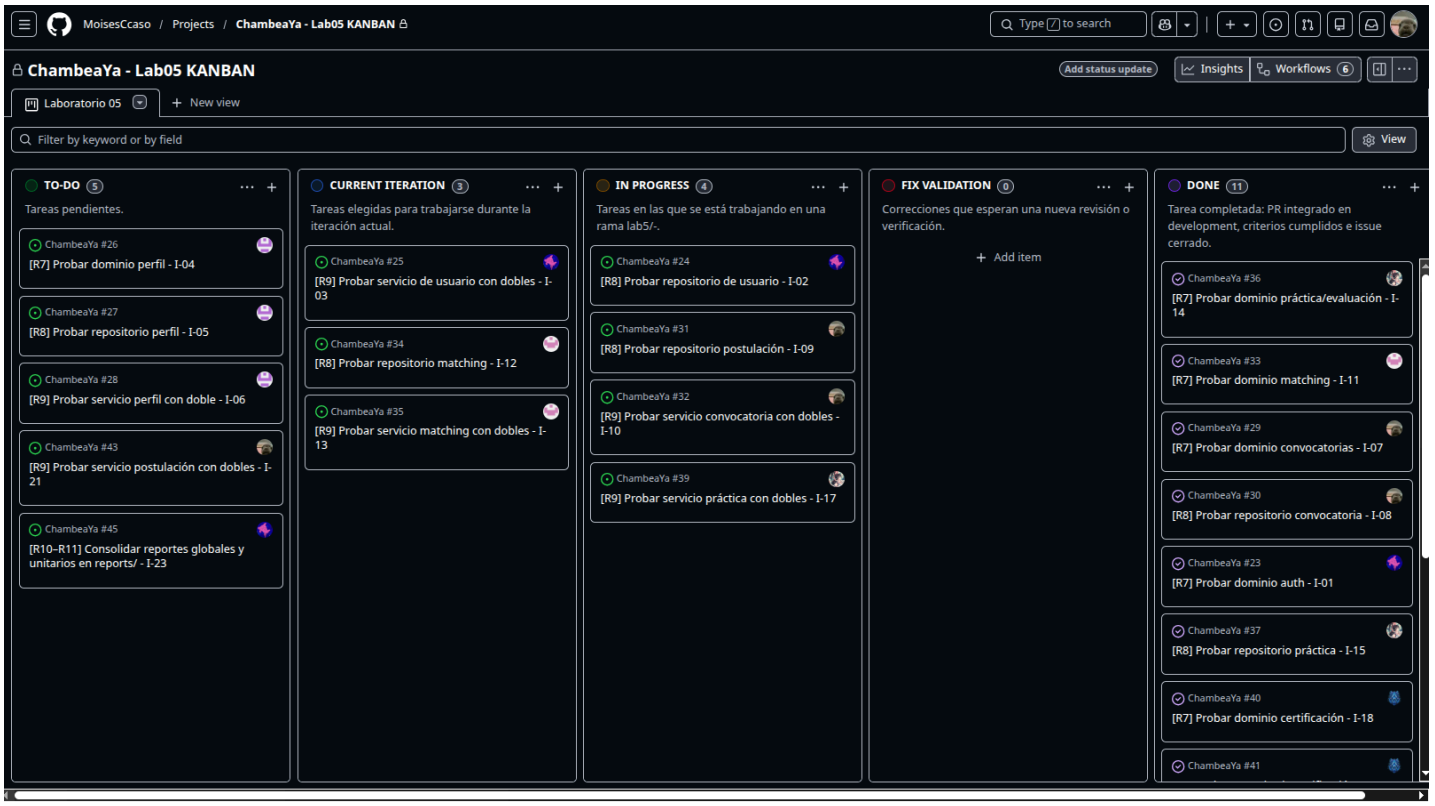


Figura 3: Creación del tablero Kanban como Github Project

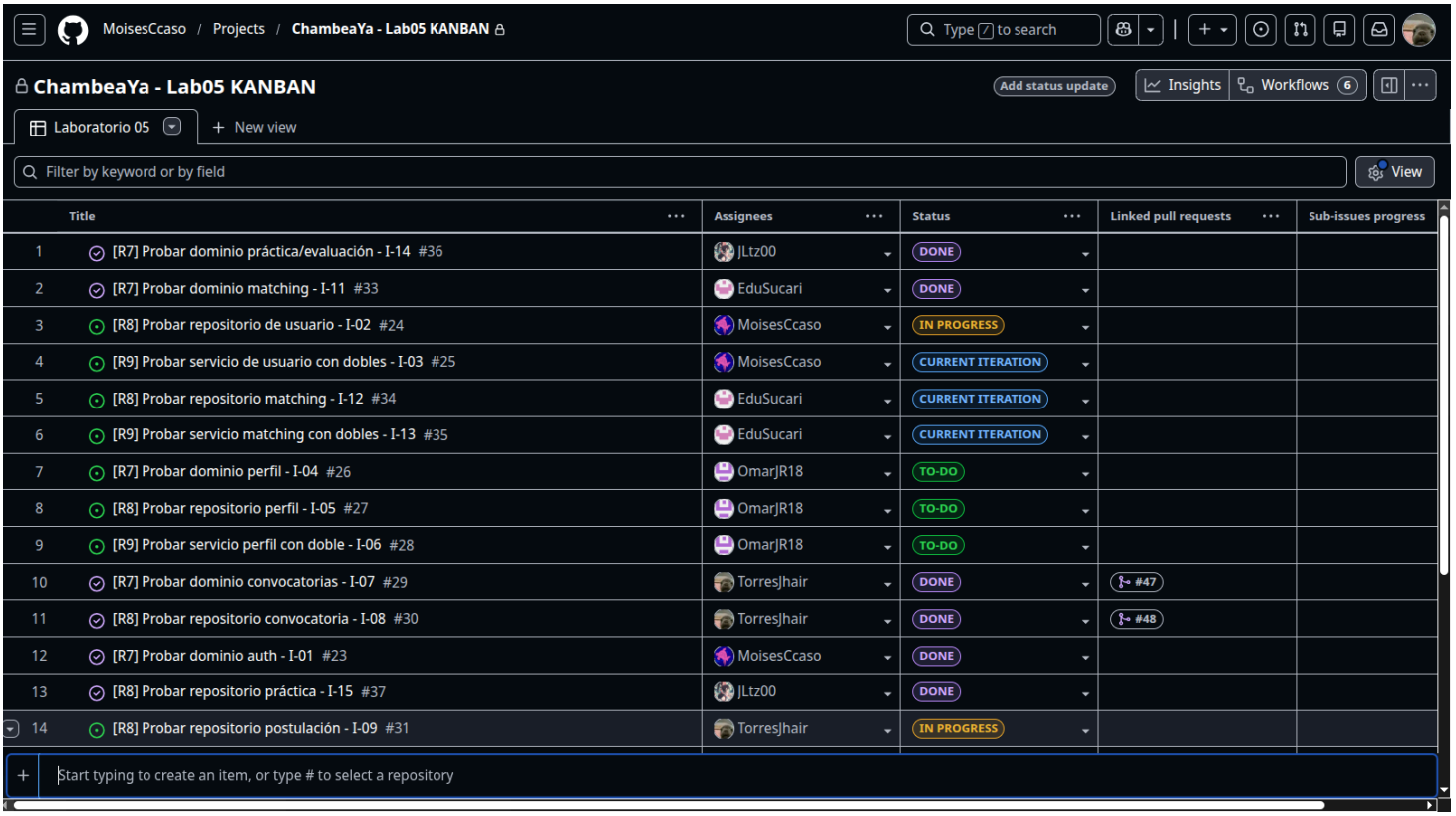


Figura 4: Vista tabular del tablero Kanban con los issues, integrante responsable, estado y Pull Request asociada

4. Pruebas del dominio

4.1. Dominio de *Práctica*

- Especificación de casos de prueba

Clase/Método	Caso de Prueba	Valores de Prueba	Resultado Esperado
`PracticaFabrica.iniciar_practica(postulacion_id, practicante_id)`	Iniciar una práctica con datos vacíos	`postulacion_id = 11, practicante_id = 22`	Retorna una Practica con los IDs indicados, estado "en curso", y listas de entregables y evaluaciones vacías
`Evaluacion.crear(practica_id, puntaje)`	Aceptar puntajes válidos y convertirlos a float	puntaje = 0, "59.5", 60, 100	puntaje = 0.0, 59.5, 60.0, 100.0; practica_id = 7 y fecha_evaluacion en UTC
`Practica.subir_entregable(archivo)`	Subir un entregable a una práctica finalizada	Práctica en estado finalizada, archivo = "otro-informe.pdf"	Lanza ValueError ("entregables...finalizada"); la lista de entregables sigue vacía

- Implementación de casos de prueba

```python
def test_inicia_practica_en_curso_y_sin_registros(self):
    practica = PracticaFabrica().iniciar_practica(11, 22)
    self.assertEqual(practica.postulacion_id, 11)
    self.assertEqual(practica.practicante_id, 22)
    self.assertEqual(practica.estado, Practica.ESTADO_EN_CURSO)
    self.assertEqual(practica.entregables, [])
    self.assertEqual(practica.evaluaciones, [])
```

```python
def test_acepta_puntajes_validos_y_los_convierte_a_float(self):
    for puntaje, esperado in ((0, 0.0), ("59.5", 59.5), (60, 60.0), (100, 100.0)):
        with self.subTest(puntaje=puntaje):
            evaluacion = Evaluacion.crear(7, puntaje)
            self.assertEqual(evaluacion.practica_id, 7)
            self.assertEqual(evaluacion.puntaje, esperado)
            self.assertEqual(evaluacion.fecha_evaluacion.tzinfo, timezone.utc)
```

```python
def test_finalizada_no_admite_nuevos_entregables_ni_evaluaciones(self):
    practica = Practica(estado=Practica.ESTADO_FINALIZADA)
    with self.assertRaisesRegex(ValueError, "entregables.*finalizada"):
        practica.subir_entregable("otro-informe.pdf")
    with self.assertRaisesRegex(ValueError, "evaluaciones.*finalizada"):
        practica.registrar_evaluacion(90)
    self.assertEqual(practica.entregables, [])
    self.assertEqual(practica.evaluaciones, [])
```

- **Resultados y reporte de cobertura de código**

Coverage report: 96%

Files | Functions | Classes

coverage.py v7.16.1, created at 2026-11-03 08:45 -0500

File	function	statements	missing	excluded	coverage
src/main/domain/practica_evaluacion/__init__.py	*(no function)*	0	0	0	100%
src/main/domain/practica_evaluacion/entregable.py	Entregable.__init__	4	0	0	100%
src/main/domain/practica_evaluacion/entregable.py	Entregable.crear	3	0	0	100%
src/main/domain/practica_evaluacion/entregable.py	*(no function)*	5	0	0	100%
src/main/domain/practica_evaluacion/evaluacion.py	Evaluacion.__init__	4	0	0	100%
src/main/domain/practica_evaluacion/evaluacion.py	Evaluacion.crear	9	0	0	100%
src/main/domain/practica_evaluacion/evaluacion.py	Evaluacion.esta_aprobada	3	0	0	100%
src/main/domain/practica_evaluacion/evaluacion.py	*(no function)*	7	0	0	100%
src/main/domain/practica_evaluacion/i_practica_repository.py	IPracticaRepository.save	1	1	0	0%
src/main/domain/practica_evaluacion/i_practica_repository.py	IPracticaRepository.find_by_id	1	1	0	0%
src/main/domain/practica_evaluacion/i_practica_repository.py	IPracticaRepository.find_by_practicante_id	1	1	0	0%
src/main/domain/practica_evaluacion/i_practica_repository.py	IPracticaRepository.find_by_postulacion_id	1	1	0	0%
src/main/domain/practica_evaluacion/i_practica_repository.py	*(no function)*	10	0	0	100%
src/main/domain/practica_evaluacion/practica_fabrica.py	PracticaFabrica.iniciar_practica	3	0	0	100%
src/main/domain/practica_evaluacion/practica_fabrica.py	*(no function)*	3	0	0	100%
src/main/domain/practica_evaluacion/practica.py	Practica.__init__	6	0	0	100%
src/main/domain/practica_evaluacion/practica.py	Practica.subir_entregable	5	0	0	100%
src/main/domain/practica_evaluacion/practica.py	Practica.registrar_evaluacion	5	0	0	100%
src/main/domain/practica_evaluacion/practica.py	Practica.eliminar_entregable	7	0	0	100%
src/main/domain/practica_evaluacion/practica.py	Practica.eliminar_evaluacion	7	0	0	100%
src/main/domain/practica_evaluacion/practica.py	Practica.finalizar	8	0	0	100%
src/main/domain/practica_evaluacion/practica.py	Practica.obtener_historial_entregables	1	0	0	100%
src/main/domain/practica_evaluacion/practica.py	Practica.obtener_historial_evaluaciones	1	0	0	100%
src/main/domain/practica_evaluacion/practica.py	*(no function)*	13	0	0	100%
Total		**108**	**4**	**0**	**96%**

coverage.py v7.16.1, created at 2026-11-03 08:45 -0500

4.2 Dominio de Matching

- Especificación de casos de prueba

Clase/Método	Caso de Prueba	Valores de Prueba	Resultado Esperado
`Sugerencia.calcular_compatibilidad(habilidades_practicante, habilidades_requeridas)`	Calcular compatibilidad cuando todas las habilidades requeridas coinciden	Habilidades del practicante: `["python", "sql"]`; requeridas: `["python", "sql"]`	Retorna `100.0`; `puntaje_match = 100.0`; habilidades coincidentes: `["python", "sql"]`.
`ResultadoMatching.es_compatible(umbral=50.0)`	Aceptar una compatibilidad igual al umbral	`score_compatibilidad = 50.0`; `umbral = 50.0`	Retorna `True`, porque el resultado alcanza el umbral.
`MatchingDominioServicio.filtrar_convocatorias(practicante, convocatorias)`	Excluir convocatorias que no están publicadas	Practicante con `["python", "sql"]`; convocatorias: una en borrador, una cerrada y una publicada, todas con requisito `["python"]`	Devuelve solo el resultado de la convocatoria publicada; se excluyen las de borrador y cerradas.

- Implementación de casos de prueba

```python
class SugerenciaCalculoTest(unittest.TestCase):
    def test_coincidencia_total_da_100(self):
        score = self.sugerencia.calcular_compatibilidad(
            ["python", "sql"], ["python", "sql"]
        )
        self.assertEqual(100.0, score)
        self.assertEqual(100.0, self.sugerencia.puntaje_match)

    def test_coincidencia_parcial(self):
        score = self.sugerencia.calcular_compatibilidad(
            ["python"], ["python", "sql"]
        )
        self.assertEqual(50.0, score)
        self.assertEqual(["python"], self.sugerencia.habilidades_coincidentes)
```

```python
class ResultadoMatchingTest(unittest.TestCase):
    def test_es_compatible_por_encima_del_umbral(self):
        self.assertTrue(ResultadoMatching(score_compatibilidad=80).es_compatible())

    def test_es_compatible_en_el_limite_exacto(self):
        self.assertTrue(ResultadoMatching(score_compatibilidad=50).es_compatible())

    def test_no_es_compatible_por_debajo_del_umbral(self):
        self.assertFalse(ResultadoMatching(score_compatibilidad=49.99).es_compatible())
```

```python
class MatchingDominioServicioFiltrarTest(unittest.TestCase):
    def test_excluye_convocatorias_sin_estado(self):
        convocatorias = [crear_convocatoria(["python"], id=1, estado=None)]
        self.assertEqual([], self.servicio.filtrar_convocatorias(self.practicante, convocat

    def test_excluye_las_que_no_alcanzan_el_umbral_por_defecto(self):
        convocatorias = [
            crear_convocatoria(["python"], id=1),                    # 100
            crear_convocatoria(["python", "java", "go"], id=2),      # 33.3
        ]
        resultados = self.servicio.filtrar_convocatorias(self.practicante, convocatorias)
        self.assertEqual([1], [r.convocatoria_id for r in resultados])
```

- **Resultados y Reporte de cobertura de Código**

Coverage report: 96%

Files | Functions | Classes

coverage.py v7.17.1, created at 2026-11-03 10:44 -0500

File	function	statements	missing	excluded	coverage
src/main/domain/matching/__init__.py	*(no function)*	0	0	0	100%
src/main/domain/matching/i_sugerencia_repository.py	ISugerenciaRepository.save	1	1	0	0%
src/main/domain/matching/i_sugerencia_repository.py	ISugerenciaRepository.find_by_practicante_id	1	1	0	0%
src/main/domain/matching/i_sugerencia_repository.py	*(no function)*	6	0	0	100%
src/main/domain/matching/matching_dominio_servicio.py	MatchingDominioServicio.calcular_match	5	0	0	100%
src/main/domain/matching/matching_dominio_servicio.py	MatchingDominioServicio.filtrar_convocatorias	8	0	0	100%
src/main/domain/matching/matching_dominio_servicio.py	*(no function)*	6	0	0	100%
src/main/domain/matching/resultado_matching.py	ResultadoMatching.__init__	5	0	0	100%
src/main/domain/matching/resultado_matching.py	ResultadoMatching.es_compatible	1	0	0	100%
src/main/domain/matching/resultado_matching.py	*(no function)*	3	0	0	100%
src/main/domain/matching/sugerencia.py	Sugerencia.__init__	5	0	0	100%
src/main/domain/matching/sugerencia.py	Sugerencia.calcular_compatibilidad	13	0	0	100%
src/main/domain/matching/sugerencia.py	*(no function)*	3	0	0	100%
Total		**57**	**2**	**0**	**96%**

coverage.py v7.17.1, created at 2026-11-03 10:44 -0500

4.3. Dominio de *Perfil*

- Especificación de casos de prueba

Clase/Método	Caso de Prueba	Valores de Prueba	Resultado Esperado
`Practicante.verificar_identidad()`	Verificar identidad con DNI válido de 8 dígitos y rechazar formato inválido	dni = "71234567"; inválidos: "1234567", "123456789", "1234567A"	Retorna el Practicante con identidad_verificada = True; para DNI inválido lanza ValueError("El DNI debe contener exactamente 8 dígitos")
`Practicante.calcular_score()`	Calcular puntaje de reputación con habilidades, formación e identidad verificada respetando el tope de 100	`habilidades = ["Python", "SQL", "Git"], formacion_educativa = ["UNSA", "Inglés"], identidad_verificada = True`	Retorna 50.0 (15 + 10 + 25) y actualiza score_reputacion = 50.0; con más de 10 habilidades y 5 formaciones respeta el tope de 100.0
`RUC.es_valido()` / `NombreCompleto.validar()`	Validar RUC según prefijo y dígito verificador SUNAT, y recortar/validar nombre completo	RUC válido: "20100070970", inválido: "20100070975"; nombres = " Ana María ", apellidos = " Torres Díaz "	RUC.es_valido() retorna True y False respectivamente; NombreCompleto.validar() recorta ambos campos y lanza ValueError si falta alguno

- Implementación de casos de prueba

```python
    def test_verificar_identidad_acepta_dni_de_exactamente_8_digitos(self):
        practicante = Practicante(dni="71234567")

        resultado = practicante.verificar_identidad()

        self.assertIs(resultado, practicante)
        self.assertTrue(practicante.identidad_verificada)

    def test_verificar_identidad_rechaza_dni_con_longitud_distinta_o_no_numerico(self):
        dnis_invalidos = ("1234567", "123456789", "1234567A", "abcdefgh")

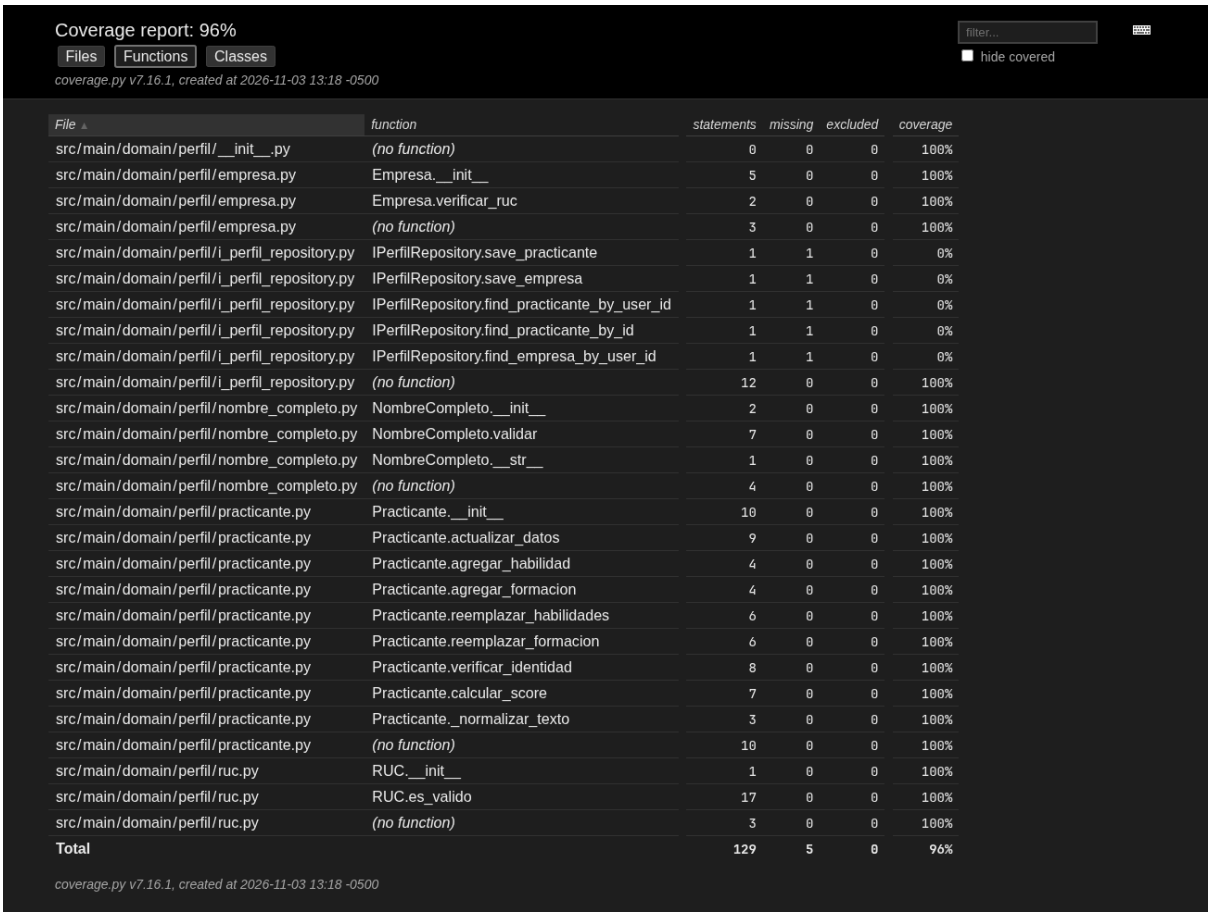
        for dni in dnis_invalidos:
            with self.subTest(dni=dni):
                practicante = Practicante(dni=dni)
                with self.assertRaisesRegex(
                    ValueError, "El DNI debe contener exactamente 8 dígitos"
                ):
                    practicante.verificar_identidad()
                self.assertFalse(practicante.identidad_verificada)
```

```python
class NombreCompletoTest(unittest.TestCase):
    def test_validar_recorta_ambos_campos_y_formatea_cadena(self):
        nombre = NombreCompleto(nombres="  Ana María  ", apellidos="  Torres Díaz  ")

        resultado = nombre.validar()

        self.assertIs(resultado, nombre)
        self.assertEqual(nombre.nombres, "Ana María")
        self.assertEqual(nombre.apellidos, "Torres Díaz")
        self.assertEqual(str(nombre), "Ana María Torres Díaz")
```

- **Resultados y Reporte de cobertura de Código**



Coverage report: 96%

Files | Functions | Classes

filter... hide covered

coverage.py v7.16.1, created at 2026-11-03 13:18 -0500

File	function	statements	missing	excluded	coverage
src/main/domain/perfil/__init__.py	*(no function)*	0	0	0	100%
src/main/domain/perfil/empresa.py	Empresa.__init__	5	0	0	100%
src/main/domain/perfil/empresa.py	Empresa.verificar_ruc	2	0	0	100%
src/main/domain/perfil/empresa.py	*(no function)*	3	0	0	100%
src/main/domain/perfil/i_perfil_repository.py	IPerfilRepository.save_practicante	1	1	0	0%
src/main/domain/perfil/i_perfil_repository.py	IPerfilRepository.save_empresa	1	1	0	0%
src/main/domain/perfil/i_perfil_repository.py	IPerfilRepository.find_practicante_by_user_id	1	1	0	0%
src/main/domain/perfil/i_perfil_repository.py	IPerfilRepository.find_practicante_by_id	1	1	0	0%
src/main/domain/perfil/i_perfil_repository.py	IPerfilRepository.find_empresa_by_user_id	1	1	0	0%
src/main/domain/perfil/i_perfil_repository.py	*(no function)*	12	0	0	100%
src/main/domain/perfil/nombre_completo.py	NombreCompleto.__init__	2	0	0	100%
src/main/domain/perfil/nombre_completo.py	NombreCompleto.validar	7	0	0	100%
src/main/domain/perfil/nombre_completo.py	NombreCompleto.__str__	1	0	0	100%
src/main/domain/perfil/nombre_completo.py	*(no function)*	4	0	0	100%
src/main/domain/perfil/practicante.py	Practicante.__init__	10	0	0	100%
src/main/domain/perfil/practicante.py	Practicante.actualizar_datos	9	0	0	100%
src/main/domain/perfil/practicante.py	Practicante.agregar_habilidad	4	0	0	100%
src/main/domain/perfil/practicante.py	Practicante.agregar_formacion	4	0	0	100%
src/main/domain/perfil/practicante.py	Practicante.reemplazar_habilidades	6	0	0	100%
src/main/domain/perfil/practicante.py	Practicante.reemplazar_formacion	6	0	0	100%
src/main/domain/perfil/practicante.py	Practicante.verificar_identidad	8	0	0	100%
src/main/domain/perfil/practicante.py	Practicante.calcular_score	7	0	0	100%
src/main/domain/perfil/practicante.py	Practicante._normalizar_texto	3	0	0	100%
src/main/domain/perfil/practicante.py	*(no function)*	10	0	0	100%
src/main/domain/perfil/ruc.py	RUC.__init__	1	0	0	100%
src/main/domain/perfil/ruc.py	RUC.es_valido	17	0	0	100%
src/main/domain/perfil/ruc.py	*(no function)*	3	0	0	100%
Total		**129**	**5**	**0**	**96%**

coverage.py v7.16.1, created at 2026-11-03 13:18 -0500

4.4. Dominio de *Auth*

- Especificación de casos de prueba

Clase/Método	Caso de Prueba	Valores de Prueba	Resultado Esperado
`Usuario.registrar()`	Registrar un usuario válido y rechazar datos obligatorios inválidos	`email = "persona@example.com", password_hash = "hash-seguro", tipo = "practicante"`; `inválidos: sin email, sin password_hash, tipo = "administrador"`	Retorna el mismo Usuario con estado "pendiente"; para los inválidos lanza ValueError("El email es obligatorio"), ValueError("La contraseña es obligatoria") y ValueError("El tipo de usuario no es válido")
`AutenticacionDominioServicio.generar_password_hash(password)`	Validar la política de contraseñas en sus límites	válidas: "Abcdef1x" (8 caracteres) y "A" + "b"×126 + "1" (128 caracteres); inválidas: None, "Abcde1x" (7), 129 caracteres, "ABCDEFG1", "abcdefg1", "Abcdefgh"	Para las válidas retorna un hash distinto de la contraseña y verificable con check_password_hash; para las inválidas lanza ValueError (con "no puede superar 128" en el caso de 129 caracteres)
`TokenRecuperacion.esta_vigente()`	Verificar la vigencia del token respecto a la fecha actual (datetime simulado)	Postulación creada para `(convocatoria_1.id, practicante_1.id)`. Búsqueda 1: `(convocatoria_1.id, practicante_1.id)`. Búsqueda 2: `(convocatoria_2.id, practicante_1.id)`	Retorna True para expiración futura y para expiración sin zona (interpretada como UTC); retorna False para token vencido, sin valor o sin expiración

- **Implementación de casos de prueba**

```python
class UsuarioTest(unittest.TestCase):
    def test_registrar_usuario_valido_devuelve_el_mismo_usuario(self):
        usuario = Usuario(
            email="persona@example.com",
            password_hash="hash-seguro",
            tipo=Usuario.TIPO_PRACTICANTE,
        )

        resultado = usuario.registrar()

        self.assertIs(resultado, usuario)
        self.assertEqual(usuario.estado, Usuario.ESTADO_PENDIENTE)

    def test_registrar_rechaza_email_ausente(self):
        usuario = Usuario(password_hash="hash-seguro", tipo=Usuario.TIPO_EMPRESA)

        with self.assertRaisesRegex(ValueError, "El email es obligatorio"):
            usuario.registrar()

    def test_registrar_rechaza_password_ausente(self):
        usuario = Usuario(email="persona@example.com", tipo=Usuario.TIPO_EMPRESA)

        with self.assertRaisesRegex(ValueError, "La contraseña es obligatoria"):
            usuario.registrar()
```

- **Resultados y Reporte de cobertura de Código**

Coverage report: 93%

Files | Functions | Classes

coverage.py v7.16.1, created at 2026-11-03 16:59 -0500

File	function	statements	missing	excluded	coverage
src/main/domain/auth/__init__.py	*(no function)*	0	0	0	100%
src/main/domain/auth/autenticacion_dominio_servicio.py	AutenticacionDominioServicio.generar_password_hash	14	0	0	100%
src/main/domain/auth/autenticacion_dominio_servicio.py	AutenticacionDominioServicio.autenticar	4	0	0	100%
src/main/domain/auth/autenticacion_dominio_servicio.py	AutenticacionDominioServicio.generar_token	2	0	0	100%
src/main/domain/auth/autenticacion_dominio_servicio.py	AutenticacionDominioServicio.validar_token	1	0	0	100%
src/main/domain/auth/autenticacion_dominio_servicio.py	*(no function)*	9	0	0	100%
src/main/domain/auth/i_auth_email_sender.py	IAuthEmailSender.send_activation	1	1	0	0%
src/main/domain/auth/i_auth_email_sender.py	IAuthEmailSender.send_password_reset	1	1	0	0%
src/main/domain/auth/i_auth_email_sender.py	IAuthEmailSender.send_test	1	1	0	0%
src/main/domain/auth/i_auth_email_sender.py	*(no function)*	8	0	0	100%
src/main/domain/auth/i_usuario_repository.py	IUsuarioRepository.save	1	1	0	0%
src/main/domain/auth/i_usuario_repository.py	IUsuarioRepository.find_by_email	1	1	0	0%
src/main/domain/auth/i_usuario_repository.py	IUsuarioRepository.find_by_id	1	1	0	0%
src/main/domain/auth/i_usuario_repository.py	IUsuarioRepository.find_by_activation_token	1	1	0	0%
src/main/domain/auth/i_usuario_repository.py	IUsuarioRepository.find_by_password_reset_token	1	1	0	0%
src/main/domain/auth/i_usuario_repository.py	*(no function)*	12	0	0	100%
src/main/domain/auth/token_recuperacion.py	TokenRecuperacion.__init__	2	0	0	100%
src/main/domain/auth/token_recuperacion.py	TokenRecuperacion.esta_vigente	6	0	0	100%
src/main/domain/auth/token_recuperacion.py	*(no function)*	4	0	0	100%
src/main/domain/auth/usuario.py	Usuario.__init__	9	0	0	100%
src/main/domain/auth/usuario.py	Usuario.registrar	7	0	0	100%
src/main/domain/auth/usuario.py	Usuario.login	3	0	0	100%
src/main/domain/auth/usuario.py	Usuario.activar	4	0	0	100%
src/main/domain/auth/usuario.py	Usuario.asignar_token_activacion	3	0	0	100%
src/main/domain/auth/usuario.py	Usuario.asignar_token_recuperacion	3	0	0	100%
src/main/domain/auth/usuario.py	Usuario.actualizar_password	6	0	0	100%
src/main/domain/auth/usuario.py	*(no function)*	12	0	0	100%
Total		**117**	**8**	**0**	**93%**

5. Pruebas de los repositorios

5.1. Repositorio de *Postulación*

- Especificación de casos de prueba

Clase/Método	Caso de Prueba	Valores de Prueba	Resultado Esperado
`SqlAlchemyPostulacionRepository.save(postulacion)`	Guardar una nueva postulación en SQLite en memoria y validar ID y estado	`convocatoria_id = convocatoria_1.id`, `practicante_id = practicante_1.id`, `estado = "pendiente"`	Retorna entidad con `id` generado (no nulo). `convocatoria_id`, `practicante_id` y `estado` persistidos correctamente
`SqlAlchemyPostulacionRepository.save(postulacion)`	Actualizar el estado de una postulación existente de `pendiente` a `seleccionada`	Postulación inicial guardada con ID generado. Modificación: `estado = "seleccionada"`	Retorna la entidad actualizada con el mismo ID. Un `find_by_id` posterior refleja el estado `"seleccionada"`
`SqlAlchemyPostulacionRepository.find_by_convocatoria_and_practicante(convocatoria_id, practicante_id)`	Buscar la postulación para una combinación dada (caso existente e inexistente)	Postulación creada para `(convocatoria_1.id, practicante_1.id)`. Búsqueda 1: `(convocatoria_1.id, practicante_1.id)`. Búsqueda 2: `(convocatoria_2.id, practicante_1.id)`	Búsqueda 1 retorna la entidad `Postulacion` coincidente. Búsqueda 2 retorna `None`

- Implementación de casos de prueba

```python
    def test_save_crea_nueva_postulacion_y_asigna_id(self):
        postulacion = Postulacion(
            convocatoria_id=self.convocatoria_1.id,
            practicante_id=self.practicante_1.id,
            estado=Postulacion.ESTADO_PENDIENTE,
        )

        guardada = self.repository.save(postulacion)

        self.assertIsNotNone(guardada.id)
        self.assertEqual(guardada.convocatoria_id, self.convocatoria_1.id)
        self.assertEqual(guardada.practicante_id, self.practicante_1.id)
        self.assertEqual(guardada.estado, Postulacion.ESTADO_PENDIENTE)

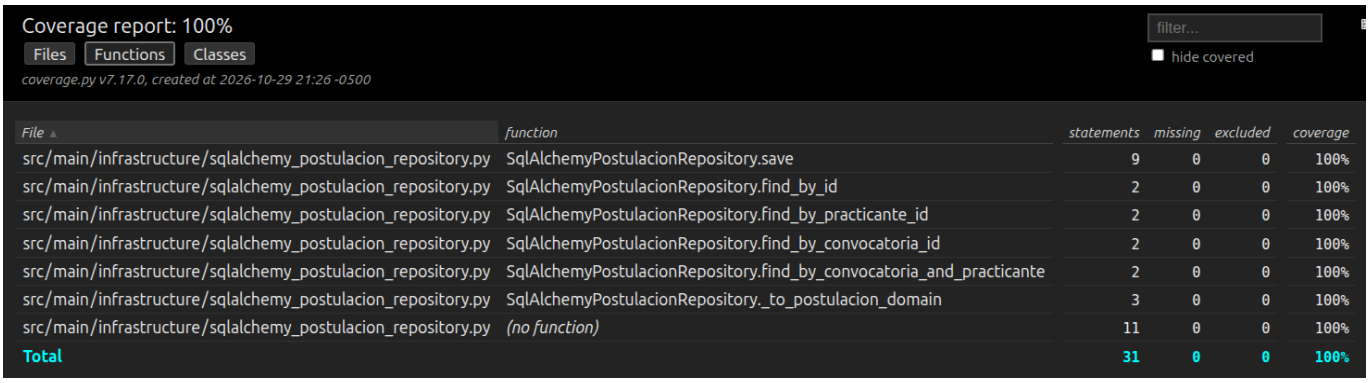
    def test_save_actualiza_estado_de_postulacion_existente(self):
        postulacion = Postulacion(
            convocatoria_id=self.convocatoria_1.id,
            practicante_id=self.practicante_1.id,
            estado=Postulacion.ESTADO_PENDIENTE,
        )
        guardada = self.repository.save(postulacion)

        guardada.estado = Postulacion.ESTADO_SELECCIONADA
        actualizada = self.repository.save(guardada)

        self.assertEqual(actualizada.id, guardada.id)
        self.assertEqual(actualizada.estado, Postulacion.ESTADO_SELECCIONADA)

        # Confirmar persistencia en BD
        recuperada = self.repository.find_by_id(guardada.id)
        self.assertEqual(recuperada.estado, Postulacion.ESTADO_SELECCIONADA)
```

- **Resultados y reporte de cobertura de código**



Coverage report: 100%

Files | Functions | Classes

filter... hide covered

coverage.py v7.17.0, created at 2026-10-29 21:26 -0500

File	function	statements	missing	excluded	coverage
src/main/infrastructure/sqlalchemy_postulacion_repository.py	SqlAlchemyPostulacionRepository.save	9	0	0	100%
src/main/infrastructure/sqlalchemy_postulacion_repository.py	SqlAlchemyPostulacionRepository.find_by_id	2	0	0	100%
src/main/infrastructure/sqlalchemy_postulacion_repository.py	SqlAlchemyPostulacionRepository.find_by_practicante_id	2	0	0	100%
src/main/infrastructure/sqlalchemy_postulacion_repository.py	SqlAlchemyPostulacionRepository.find_by_convocatoria_id	2	0	0	100%
src/main/infrastructure/sqlalchemy_postulacion_repository.py	SqlAlchemyPostulacionRepository.find_by_convocatoria_and_practicante	2	0	0	100%
src/main/infrastructure/sqlalchemy_postulacion_repository.py	SqlAlchemyPostulacionRepository._to_postulacion_domain	3	0	0	100%
src/main/infrastructure/sqlalchemy_postulacion_repository.py	*(no function)*	11	0	0	100%
Total		**31**	**0**	**0**	**100%**

5.2. Repositorio de *Práctica*

- Especificación de casos de prueba

Clase/Método	Caso de Prueba	Valores de Prueba	Resultado Esperado
`SqlAlchemyPracticaRepository.save(practica)`	Guardar una práctica nueva con un entregable y una evaluación en SQLite en memoria	postulacion_id y practicante_id válidos, entregable "informe.pdf", evaluación con puntaje = 85	Retorna la práctica con `id` generado y estado "en curso". El entregable y la evaluación tienen `id` y `practica_id` asignados. Los datos quedan persistidos
SqlAlchemyPracticaRepository.save(practica)	Actualizar una práctica existente agregando hijos nuevos sin duplicar los anteriores	Práctica guardada con "primer-informe.pdf" y puntaje 65. Luego se agregan "informe-final.pdf" y puntaje 90, y estado = "finalizada"	Estado "finalizada" y 2 entregables y 2 evaluaciones en el orden agregado. En BD hay 1 práctica, 2 entregables y 2 evaluaciones
SqlAlchemyPracticaRepository.find_by_id(practica_id)	Reconstruir el agregado completo (caso existente e inexistente)	Búsqueda 1: id de práctica guardada con "evidencia.zip" y puntaje 72. Búsqueda 2: id = 9999	Búsqueda 1 retorna una `Practica` con los mismos IDs, estado "en curso", entregable "evidencia.zip" y evaluación 72. Búsqueda 2 retorna `None`

- **Implementación de casos de prueba**

```python
def test_save_persiste_practica_entregable_y_evaluacion(
    app_context,
    repository,
    datos_practica,
):
    fecha = datetime(2026, 10, 28, 15, 30, tzinfo=timezone.utc)
    practica = crear_practica(
        datos_practica,
        entregables=[Entregable(archivo="informe.pdf", fecha_subida=fecha)],
        evaluaciones=[Evaluacion(puntaje=85, fecha_evaluacion=fecha)],
    )

    guardada = repository.save(practica)
    db.session.expire_all()
Ctrl U Side AI Chat, Ctrl I Inline AI Chat
    modelo = db.session.get(PracticaModel, guardada.id)
    assert modelo is not None
    assert modelo.postulacion_id == datos_practica["postulacion_id"]
    assert modelo.practicante_id == datos_practica["practicante_id"]
    assert modelo.estado == Practica.ESTADO_EN_CURSO
    assert len(modelo.entregables) == 1
    assert modelo.entregables[0].archivo == "informe.pdf"
    assert len(modelo.evaluaciones) == 1
    assert modelo.evaluaciones[0].puntaje == 85

    assert guardada.id is not None
    assert guardada.entregables[0].id is not None
    assert guardada.entregables[0].practica_id == guardada.id
    assert guardada.evaluaciones[0].id is not None
    assert guardada.evaluaciones[0].practica_id == guardada.id
```

```python
def test_save_actualiza_estado_y_agrega_nuevos_hijos_sin_duplicar_existentes(
    app_context,
    repository,
    datos_practica,
):
    practica = crear_practica(datos_practica)
    practica.subir_entregable("primer-informe.pdf")
    practica.registrar_evaluacion(65)
    practica = repository.save(practica)

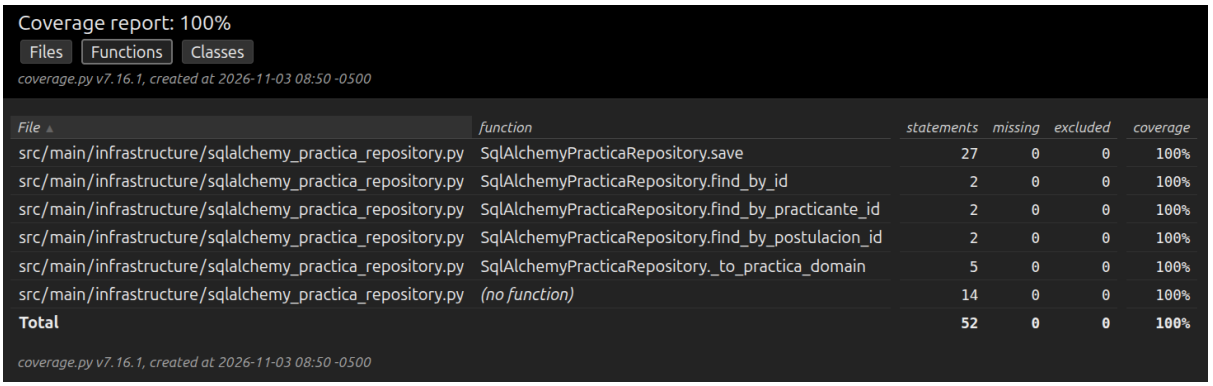
    practica.subir_entregable("informe-final.pdf")
    practica.registrar_evaluacion(90)
    practica.estado = Practica.ESTADO_FINALIZADA
    actualizada = repository.save(practica)

    assert actualizada.estado == Practica.ESTADO_FINALIZADA
    assert [item.archivo for item in actualizada.entregables] == [
        "primer-informe.pdf",
        "informe-final.pdf",
    ]
    assert [item.puntaje for item in actualizada.evaluaciones] == [65, 90]
    assert PracticaModel.query.count() == 1
    assert EntregableModel.query.count() == 2
    assert EvaluacionModel.query.count() == 2
```

```python
def test_find_by_id_reconstruye_el_agregado_completo(
    app_context,
    repository,
    datos_practica,
):
    practica = crear_practica(datos_practica)
    practica.subir_entregable("evidencia.zip")
    practica.registrar_evaluacion(72)
    guardada = repository.save(practica)
Ctrl U Side AI Chat, Ctrl I Inline AI Chat
    encontrada = repository.find_by_id(guardada.id)

    assert isinstance(encontrada, Practica)
    assert encontrada.id == guardada.id
    assert encontrada.postulacion_id == datos_practica["postulacion_id"]
    assert encontrada.practicante_id == datos_practica["practicante_id"]
    assert encontrada.estado == Practica.ESTADO_EN_CURSO
    assert [item.archivo for item in encontrada.entregables] == ["evidencia.zip"]
    assert [item.puntaje for item in encontrada.evaluaciones] == [72]
```

- Resultados y reporte de cobertura de código



Coverage report: 100%

Files | Functions | Classes

coverage.py v7.16.1, created at 2026-11-03 08:50 -0500

File	function	statements	missing	excluded	coverage
src/main/infrastructure/sqlalchemy_practica_repository.py	SqlAlchemyPracticaRepository.save	27	0	0	100%
src/main/infrastructure/sqlalchemy_practica_repository.py	SqlAlchemyPracticaRepository.find_by_id	2	0	0	100%
src/main/infrastructure/sqlalchemy_practica_repository.py	SqlAlchemyPracticaRepository.find_by_practicante_id	2	0	0	100%
src/main/infrastructure/sqlalchemy_practica_repository.py	SqlAlchemyPracticaRepository.find_by_postulacion_id	2	0	0	100%
src/main/infrastructure/sqlalchemy_practica_repository.py	SqlAlchemyPracticaRepository._to_practica_domain	5	0	0	100%
src/main/infrastructure/sqlalchemy_practica_repository.py	*(no function)*	14	0	0	100%
Total		**52**	**0**	**0**	**100%**

coverage.py v7.16.1, created at 2026-11-03 08:50 -0500

5.3. Repositorio de *Perfil*

- Especificación de casos de prueba

Clase/Método	Caso de Prueba	Valores de Prueba	Resultado Esperado
`SqlAlchemyPerfilRepository.save_practicante(practicante)`	Guardar y recargar perfil de practicante en SQLite en memoria con listas y banderas	`usuario_id válido, dni = "71234567", habilidades = ["Python", "SQL", "Flask"], score_reputacion = 50.0, identidad_verificada = True`	Retorna Practicante con id generado; al consultar con `find_practicante_by_user_id` recupera listas deserializadas y banderas intactas
`SqlAlchemyPerfilRepository.save_empresa(empresa)`	Guardar y recuperar empresa con objeto de valor RUC encapsulado	`usuario_id válido, razon_social = "Innovación Digital S.A.C.", ruc = RUC("20100070970"), verificada = True`	Retorna y recupera Empresa con id asignado, `verificada = True` y ruc reconstruido como instancia de RUC("20100070970")
`SqlAlchemyPerfilRepository.find_practicante_by_user_id(usuario_id)`	Consultar perfil de practicante y empresa con identificadores inexistentes	`usuario_id = 99999, practicante_id = 99999`	`find_practicante_by_user_id`, `find_practicante_by_id` y `find_empresa_by_user_id` retornan `None`

- **Implementación de casos de prueba**

```python
    def test_guardar_y_recargar_perfil_de_practicante_listas_y_banderas(self):
        usuario = self._crear_usuario_en_db("practicante1@test.local", tipo="practicante")
        practicante = Practicante(
            usuario_id=usuario.id,
            nombres="Lucía",
            apellidos="Mendoza",
            dni="71234567",
            carnet_universitario="20210987",
            habilidades=["Python", "SQL", "Flask"],
            formacion_educativa=["UNSA - Ciencia de la Computación", "Inglés B2"],
            score_reputacion=50.0,
            identidad_verificada=True,
        )

        guardado = self.repository.save_practicante(practicante)

        self.assertIsNotNone(guardado.id)
        self.assertEqual(guardado.usuario_id, usuario.id)
        self.assertEqual(guardado.nombres, "Lucía")
        self.assertEqual(guardado.apellidos, "Mendoza")
```

```python
    def test_guardar_y_recuperar_empresa_con_ruc_encapsulado(self):
        usuario = self._crear_usuario_en_db("empresa1@test.local", tipo="empresa")
        empresa = Empresa(
            usuario_id=usuario.id,
            razon_social="Innovación Digital S.A.C.",
            ruc=RUC("20100070970"),
            verificada=True,
        )

        guardada = self.repository.save_empresa(empresa)

        self.assertIsNotNone(guardada.id)
        self.assertEqual(guardada.usuario_id, usuario.id)
        self.assertEqual(guardada.razon_social, "Innovación Digital S.A.C.")
        self.assertIsInstance(guardada.ruc, RUC)
        self.assertEqual(guardada.ruc.numero, "20100070970")
        self.assertTrue(guardada.ruc.es_valido())
        self.assertTrue(guardada.verificada)

        recuperada = self.repository.find_empresa_by_user_id(usuario.id)
```

```python
    def test_busquedas_inexistentes_devuelven_none(self):
        self.assertIsNone(self.repository.find_practicante_by_user_id(99999))
        self.assertIsNone(self.repository.find_practicante_by_id(99999))
        self.assertIsNone(self.repository.find_empresa_by_user_id(99999))
```

- **Resultados y reporte de cobertura de código**

Coverage report: 100%

Files | Functions | Classes

filter... hide covered

coverage.py v7.16.1, created at 2026-11-03 13:18 -0500

File	function	statements	missing	excluded	coverage
src/main/infrastructure/sqlalchemy_perfil_repository.py	SqlAlchemyPerfilRepository.save_practicante	18	0	0	100%
src/main/infrastructure/sqlalchemy_perfil_repository.py	SqlAlchemyPerfilRepository.save_empresa	13	0	0	100%
src/main/infrastructure/sqlalchemy_perfil_repository.py	SqlAlchemyPerfilRepository.find_practicante_by_user_id	2	0	0	100%
src/main/infrastructure/sqlalchemy_perfil_repository.py	SqlAlchemyPerfilRepository.find_practicante_by_id	2	0	0	100%
src/main/infrastructure/sqlalchemy_perfil_repository.py	SqlAlchemyPerfilRepository.find_empresa_by_user_id	2	0	0	100%
src/main/infrastructure/sqlalchemy_perfil_repository.py	SqlAlchemyPerfilRepository._to_practicante_domain	3	0	0	100%
src/main/infrastructure/sqlalchemy_perfil_repository.py	SqlAlchemyPerfilRepository._to_empresa_domain	3	0	0	100%
src/main/infrastructure/sqlalchemy_perfil_repository.py	SqlAlchemyPerfilRepository._dump_list	1	0	0	100%
src/main/infrastructure/sqlalchemy_perfil_repository.py	SqlAlchemyPerfilRepository._load_list	9	0	0	100%
src/main/infrastructure/sqlalchemy_perfil_repository.py	*(no function)*	18	0	0	100%
Total		**71**	**0**	**0**	**100%**

coverage.py v7.16.1, created at 2026-11-03 13:18 -0500

5.4. Repositorio de *Usuario*

- Especificación de casos de prueba

Clase/Método	Caso de Prueba	Valores de Prueba	Resultado Esperado
`SqlAlchemyUsuarioRepository.save(usuario) / find_by_email(email)`	Guardar un usuario nuevo en SQLite en memoria y recuperarlo por email	`email = "ana@test.local", password_hash = "hash-prueba", tipo = "practicante", estado = "pendiente"`	save retorna el Usuario con id generado; find_by_email retorna un Usuario con el mismo id, email, tipo y estado
`SqlAlchemyUsuarioRepository.save(usuario) / find_by_activation_token(token) / find_by_password_reset_token(token)`	Guardar los tokens de activación y de recuperación y consultarlos	`activación: "tok-activar" con expiración a 48 h; recuperación: "tok-recuperar" con expiración a 2 h`	Cada búsqueda retorna el usuario guardado con su token y la expiración persistida (sin zona horaria)
`SqlAlchemyUsuarioRepository.find_by_email(email) / find_by_id(id) / find_by_activation_token(token) / find_by_password_reset_token(token)`	Consultar con valores inexistentes	`email = "no-existe@test.local", id = 99999, token = "token-inexistente"`	Todas las consultas retornan None

- **Implementación de casos de prueba**

src > test > infrastructure > test_usuario_repository.py > test_guardar_y_buscar_usuario_por_identificador

```python

def test_guardar_y_buscar_usuario_por_email(app_context):
    repository = SqlAlchemyUsuarioRepository()
    usuario = repository.save(usuario_de_prueba(email="ana@test.local"))

    encontrado = repository.find_by_email("ana@test.local")

    assert encontrado is not None
    assert encontrado.id == usuario.id
    assert encontrado.email == "ana@test.local"
    assert encontrado.tipo == Usuario.TIPO_PRACTICANTE
    assert encontrado.estado == Usuario.ESTADO_PENDIENTE


def test_guardar_y_buscar_usuario_por_identificador(app_context):
    repository = SqlAlchemyUsuarioRepository()
    usuario = repository.save(usuario_de_prueba(email="juan@test.local"))

    encontrado = repository.find_by_id(usuario.id)

    assert encontrado is not None
    assert encontrado.id == usuario.id
    assert encontrado.email == "juan@test.local"


def test_buscar_email_inexistente_devuelve_none(app_context):
    repository = SqlAlchemyUsuarioRepository()

    assert repository.find_by_email("no-existe@test.local") is None
    assert repository.find_by_id(99999) is None
```

- **Resultados y reporte de cobertura de código**

Coverage report: 97%

Files | Functions | Classes

coverage.py v7.16.1, created at 2026-11-03 17:00 -0500

File	function	statements	missing	excluded	coverage
src/main/infrastructure/sqlalchemy_usuario_repository.py	SqlAlchemyUsuarioRepository.save	16	1	0	94%
src/main/infrastructure/sqlalchemy_usuario_repository.py	SqlAlchemyUsuarioRepository.find_by_email	2	0	0	100%
src/main/infrastructure/sqlalchemy_usuario_repository.py	SqlAlchemyUsuarioRepository.find_by_id	2	0	0	100%
src/main/infrastructure/sqlalchemy_usuario_repository.py	SqlAlchemyUsuarioRepository.find_by_activation_token	2	0	0	100%
src/main/infrastructure/sqlalchemy_usuario_repository.py	SqlAlchemyUsuarioRepository.find_by_password_reset_token	2	0	0	100%
src/main/infrastructure/sqlalchemy_usuario_repository.py	SqlAlchemyUsuarioRepository._to_domain	3	0	0	100%
src/main/infrastructure/sqlalchemy_usuario_repository.py	*(no function)*	11	0	0	100%
Total		**38**	**1**	**0**	**97%**

coverage.py v7.16.1, created at 2026-11-03 17:00 -0500

6. Pruebas de los servicios de aplicación

6.1. Servicio de *Convocatorias* (Dobles de prueba)

- Especificación de casos de prueba

Clase/Método	Caso de Prueba	Valores de Prueba	Resultado Esperado
`ConvocatoriaApplicationService.publish(empresa_id, convocatoria_id)`	Publicar una convocatoria propia existente en estado borrador	`empresa_id = 1`, `convocatoria_id = 10`. Mock: `Convocatoria(id=10, empresa_id=1, estado="borrador")`	Estado cambia a `"publicada"`. Se llama a `find_by_id(10)` y a `save(convocatoria)`. Retorna la entidad guardada con estado `"publicada"`
`ConvocatoriaApplicationService.publish(empresa_id, convocatoria_id)`	Control de acceso: publicar una convocatoria de otra empresa	`empresa_id = 1`, `convocatoria_id = 10`. Mock: `Convocatoria(id=10, empresa_id=2, estado="borrador")`	Lanza `ValueError` con "La convocatoria no pertenece a esta empresa". `save(...)` no es invocado
`ConvocatoriaApplicationService.search(query, estado)`	Delegar filtros de búsqueda (texto y estado) al repositorio	`query = "Python"`, `estado = "publicada"`. Mock retorna `[Convocatoria(id=1), Convocatoria(id=2)]`	El repositorio recibe `search(query="Python", estado="publicada")`. Retorna la misma lista del mock

- **Implementación de casos de prueba**

```python
    def test_publish_convocatoria_de_otra_empresa_lanza_error(self):
        convocatoria = Convocatoria(id=10, empresa_id=2, estado=Convocatoria.ESTADO_BOR
        self.mock_repo.find_by_id.return_value = convocatoria

        with self.assertRaises(ValueError) as ctx:
            self.service.publish(empresa_id=1, convocatoria_id=10)
        self.assertIn("La convocatoria no pertenece a esta empresa", str(ctx.exception)
        self.mock_repo.save.assert_not_called()

    def test_publish_exitoso_cambia_a_publicada_y_guarda(self):
        convocatoria = Convocatoria(id=10, empresa_id=1, estado=Convocatoria.ESTADO_BOR
        self.mock_repo.find_by_id.return_value = convocatoria
        self.mock_repo.save.return_value = convocatoria

        resultado = self.service.publish(empresa_id=1, convocatoria_id=10)

        self.assertEqual(resultado.estado, Convocatoria.ESTADO_PUBLICADA)
        self.mock_repo.find_by_id.assert_called_once_with(10)
        self.mock_repo.save.assert_called_once_with(convocatoria)
```

- **Resultados y reporte de cobertura de código**

Coverage report: 100%

Files | Functions | Classes

filter...

hide covered

coverage.py v7.17.0, created at 2026-10-29 21:58 -0500

File	function	statements	missing	excluded	coverage
src/main/application/convocatoria_application_service.py	ConvocatoriaApplicationService.__init__	2	0	0	100%
src/main/application/convocatoria_application_service.py	ConvocatoriaApplicationService.create_convocatoria	7	0	0	100%
src/main/application/convocatoria_application_service.py	ConvocatoriaApplicationService.publish	8	0	0	100%
src/main/application/convocatoria_application_service.py	ConvocatoriaApplicationService.update	4	0	0	100%
src/main/application/convocatoria_application_service.py	ConvocatoriaApplicationService.close	4	0	0	100%
src/main/application/convocatoria_application_service.py	ConvocatoriaApplicationService.reopen	4	0	0	100%
src/main/application/convocatoria_application_service.py	ConvocatoriaApplicationService.duplicate	3	0	0	100%
src/main/application/convocatoria_application_service.py	ConvocatoriaApplicationService.delete	6	0	0	100%
src/main/application/convocatoria_application_service.py	ConvocatoriaApplicationService.search	2	0	0	100%
src/main/application/convocatoria_application_service.py	ConvocatoriaApplicationService.find_by_id	5	0	0	100%
src/main/application/convocatoria_application_service.py	ConvocatoriaApplicationService.list_for_empresa	2	0	0	100%
src/main/application/convocatoria_application_service.py	ConvocatoriaApplicationService._find_owned	6	0	0	100%
src/main/application/convocatoria_application_service.py	ConvocatoriaApplicationService._require_repository	2	0	0	100%
src/main/application/convocatoria_application_service.py	*(no function)*	15	0	0	100%
Total		**70**	**0**	**0**	**100%**

coverage.py v7.17.0, created at 2026-10-29 21:58 -0500

6.2 Servicio de Matching (Dobles de Prueba)

- Especificación de casos de Prueba

Clase/Método	Caso de Prueba	Valores de Prueba	Resultado Esperado
`MatchingApplicationService.suggest_convocatorias(usuario_id, umbral)`	Obtener convocatorias compatibles aplicando un umbral personalizado	`usuario_id = 5`; `umbral = 65.0`; practicante y convocatorias devueltos por sus dobles	Devuelve la lista de resultados del servicio de dominio. Consulta el perfil y las convocatorias, y llama a `filtrar_convocatorias(practicante, convocatorias, 65.0)`. No guarda sugerencias.
`MatchingApplicationService.calculate_for_practicante(usuario_id)`	Calcular coincidencias y guardar las sugerencias obtenidas	`usuario_id = 5`; dos resultados con sus respectivas sugerencias	Devuelve los dos resultados y llama a `sugerencia_repository.save()` una vez por cada sugerencia.
`MatchingApplicationService.suggest_convocatorias(usuario_id, umbral)`	Manejar un usuario que no tiene perfil de practicante	`usuario_id = 404`; el repositorio de perfil devuelve `None`	Lanza `ValueError("Practicante no encontrado")`. No consulta convocatorias ni invoca el servicio de matching.

- **Implementación de Casos de Prueba**

```python
def test_suggest_convocatorias_devuelve_resultados_y_consulta_dependencias(self):
    resultados = self.service.suggest_convocatorias(usuario_id=5, umbral=65.0)

    self.assertIs(self.resultados, resultados)
    self.perfil_repository.find_practicante_by_user_id.assert_called_once_with(5)
    self.convocatoria_repository.find_all.assert_called_once_with()
    self.matching_servicio.filtrar_convocatorias.assert_called_once_with(
        self.practicante, self.convocatorias, 65.0
    )
    self.sugerencia_repository.save.assert_not_called()
```

```python
def test_calculate_for_practicante_guarda_cada_sugerencia_y_devuelve_resultados(self):
    resultados = self.service.calculate_for_practicante(usuario_id=5)

    self.assertIs(self.resultados, resultados)
    self.perfil_repository.find_practicante_by_user_id.assert_called_once_with(5)
    self.convocatoria_repository.find_all.assert_called_once_with()
    self.matching_servicio.filtrar_convocatorias.assert_called_once_with(
        self.practicante, self.convocatorias
    )
    self.sugerencia_repository.save.assert_has_calls(
        [call(self.resultado_1.sugerencia), call(self.resultado_2.sugerencia)]
    )
    self.assertEqual(2, self.sugerencia_repository.save.call_count)
```

```python
def test_lanza_error_si_no_encuentra_al_practicante(self):
    self.perfil_repository.find_practicante_by_user_id.return_value = None

    with self.assertRaisesRegex(ValueError, "Practicante no encontrado"):
        self.service.suggest_convocatorias(usuario_id=404)

    self.perfil_repository.find_practicante_by_user_id.assert_called_once_with(404)
    self.convocatoria_repository.find_all.assert_not_called()
    self.matching_servicio.filtrar_convocatorias.assert_not_called()
```

- **Resultados y reporte de cobertura de código**

Coverage report: 90%

Files | Functions | Classes

coverage.py v7.17.1, created at 2026-11-03 11:47 -0500

File ▲	function	statements	missing	excluded	coverage
src/main/application/matching_application_service.py	MatchingApplicationService.__init__	4	0	0	100%
src/main/application/matching_application_service.py	MatchingApplicationService.suggest_convocatorias	6	0	0	100%
src/main/application/matching_application_service.py	MatchingApplicationService.calculate_for_practicante	9	1	0	89%
src/main/application/matching_application_service.py	MatchingApplicationService._require_repositories	6	2	0	67%
src/main/application/matching_application_service.py	*(no function)*	6	0	0	100%
Total		**31**	**3**	**0**	**90%**

6.3 Servicio de Perfil (Dobles de Prueba)

- Especificación de casos de Prueba

Clase/Método	Caso de Prueba	Valores de Prueba	Resultado Esperado
`PerfilApplicationService.update_practicante(usuario_id, data)`	Actualizar perfil de practicante activo verificando identidad, score y guardado en el doble	`usuario_id = 10 (activo); dni = " 71234567 ", habilidades = ["Python", "SQL"], formacion_educativa = ["UNSA", "Inglés"]`	Normaliza DNI, marca identidad_verificada = True, calcula score_reputacion = 45.0 y llama a save_practicante(practicante) una vez
`PerfilApplicationService.update_practicante(usuario_id, data)`	Control de acceso: rechazar usuario inexistente, inactivo o de tipo empresa antes de guardar	`usuario_id = 99 (None), usuario con estado = "pendiente", y usuario con tipo = "empresa"`	Lanza ValueError en cada caso ("Usuario no encontrado", "La cuenta no está activa", "El usuario no es practicante"); save_practicante no es invocado
`PerfilApplicationService.update_empresa(usuario_id, data)`	Actualizar perfil de empresa activa validando razón social y RUC	`usuario_id = 20 (empresa activa); razon_social = " Acme S.A.C. ", ruc = "20100070970"`	Recorta razon_social, valida RUC (verificada = True) e invoca save_empresa(empresa) una vez; rechaza RUC inválido con ValueError

- Implementación de Casos de Prueba

```python
def test_update_practicante_activo_crea_o_actualiza_verifica_identidad_y_guarda(self):
    usuario = self._usuario_practicante_activo(usuario_id=10)
    self.usuario_repository.find_by_id.return_value = usuario
    self.perfil_repository.find_practicante_by_user_id.return_value = None
    self.perfil_repository.save_practicante.side_effect = lambda p: p

    datos = {
        "nombres": "Ana",
        "apellidos": "Torres",
        "dni": "  71234567  ",
        "carnet_universitario": "  20214567  ",
        "habilidades": [" Python ", "SQL", "Python"],
        "formacion_educativa": [" UNSA ", "Inglés"],
    }

    resultado = self.service.update_practicante(10, datos)

    self.usuario_repository.find_by_id.assert_called_once_with(10)
    self.perfil_repository.find_practicante_by_user_id.assert_called_once_with(10)
    self.perfil_repository.save_practicante.assert_called_once()
```

```python
def test_update_practicante_rechaza_usuario_inactivo_antes_de_guardar(self):
    usuario_inactivo = Usuario(
        id=10,
        email="pendiente@test.local",
        password_hash="hash",
        tipo=Usuario.TIPO_PRACTICANTE,
        estado=Usuario.ESTADO_PENDIENTE,
    )
    self.usuario_repository.find_by_id.return_value = usuario_inactivo

    with self.assertRaisesRegex(ValueError, "La cuenta no está activa"):
        self.service.update_practicante(10, {"nombres": "Ana"})

    self.usuario_repository.find_by_id.assert_called_once_with(10)
    self.perfil_repository.find_practicante_by_user_id.assert_not_called()
    self.perfil_repository.save_practicante.assert_not_called()
```

```python
139     def test_update_practicante_rechaza_usuario_tipo_empresa_antes_de_guardar(self):
140         usuario_empresa = self._usuario_empresa_activo(usuario_id=20)
141         self.usuario_repository.find_by_id.return_value = usuario_empresa
142
143         with self.assertRaisesRegex(ValueError, "El usuario no es practicante"):
144             self.service.update_practicante(20, {"nombres": "Ana"})
145
146         self.usuario_repository.find_by_id.assert_called_once_with(20)
147         self.perfil_repository.find_practicante_by_user_id.assert_not_called()
148         self.perfil_repository.save_practicante.assert_not_called()
149
```

- **Resultados y reporte de cobertura de código**

Coverage report: 100%

Files | Functions | Classes

filter... hide covered

coverage.py v7.16.1, created at 2026-11-03 13:19 -0500

File	function	statements	missing	excluded	coverage
src/main/application/perfil_application_service.py	PerfilApplicationService.__init__	2	0	0	100%
src/main/application/perfil_application_service.py	PerfilApplicationService.get_practicante_profile	3	0	0	100%
src/main/application/perfil_application_service.py	PerfilApplicationService.update_practicante	21	0	0	100%
src/main/application/perfil_application_service.py	PerfilApplicationService.add_habilidad	5	0	0	100%
src/main/application/perfil_application_service.py	PerfilApplicationService.add_formacion	5	0	0	100%
src/main/application/perfil_application_service.py	PerfilApplicationService.register_identity	6	0	0	100%
src/main/application/perfil_application_service.py	PerfilApplicationService.update_empresa	15	0	0	100%
src/main/application/perfil_application_service.py	PerfilApplicationService.get_empresa_profile	3	0	0	100%
src/main/application/perfil_application_service.py	PerfilApplicationService.verify_profile	5	0	0	100%
src/main/application/perfil_application_service.py	PerfilApplicationService.get_reputation_score	5	0	0	100%
src/main/application/perfil_application_service.py	PerfilApplicationService._get_or_create_practicante	5	0	0	100%
src/main/application/perfil_application_service.py	PerfilApplicationService._require_practicante_user	8	0	0	100%
src/main/application/perfil_application_service.py	PerfilApplicationService._require_empresa_user	8	0	0	100%
src/main/application/perfil_application_service.py	PerfilApplicationService._require_repositories	4	0	0	100%
src/main/application/perfil_application_service.py	PerfilApplicationService._optional_text	4	0	0	100%
src/main/application/perfil_application_service.py	*(no function)*	20	0	0	100%
Total		**119**	**0**	**0**	**100%**

coverage.py v7.16.1, created at 2026-11-03 13:19 -0500

6.4 Servicio de *Usuario* (Dobles de Prueba)

- Especificación de casos de Prueba

Clase/Método	Caso de Prueba	Valores de Prueba	Resultado Esperado
`UsuarioApplicationService.register_user(email, password, password_confirmation, tipo)`	Registrar un usuario normalizando el email, guardándolo como pendiente y enviando el correo de activación	`email = “ Ana.PEREZ@Example.COM “, password = “Abcdefg1”, confirmación = “Abcdefg1”, tipo = “practicante”. Mock: find_by_email retorna None`	Retorna Usuario con email = “ana.perez@example.com”, estado “pendiente” y token de activación con expiración a 48 h. Se llama a find_by_email("ana.perez@example.com") una vez, a save una vez y a send_activation(email, token) una vez
`UsuarioApplicationService.register_user(email, password, password_confirmation, tipo)`	Control de duplicados: rechazar un correo ya registrado	`email = “ana@test.local”, tipo = “empresa”. Mock: find_by_email retorna un Usuario existente`	Lanza ValueError con “ya está registrado”. save y send_activation no son invocados
`UsuarioApplicationService.activate_account(token)`	Activar cuenta con token inexistente, expirado y válido	`token inexistente (mock retorna None); token expirado (expiración 2020-01-01); token válido (expiración a 1 h)`	Lanza ValueError(“Token de activación inválido”) y ValueError(“Token de activación expirado”), sin invocar save; con token válido el estado pasa a “activo”, activation_token queda en None y se llama a save(usuario) una vez

- **Implementación de Casos de Prueba**

src > test > application > test_usuario_service.py > TestUsuarioService > test_registro_normaliza_email_guarda_usuario_pendiente_y_envia_activacion

```python
class TestUsuarioService(unittest.TestCase):
    def test_registro_normaliza_email_guarda_usuario_pendiente_y_envia_activacion(self):
        self.repositorio.find_by_email.return_value = None
        self.repositorio.save.side_effect = lambda usuario: usuario

        resultado = self.service.register_user(
            "  Ana.PEREZ@Example.COM ", "Abcdefg1", "Abcdefg1", Usuario.TIPO_PRACTICANTE,
        )

        self.assertEqual(resultado.email, "ana.perez@example.com")
        self.assertEqual(resultado.estado, Usuario.ESTADO_PENDIENTE)
        self.assertIsNotNone(resultado.activation_token)
        self.assertEqual(
            resultado.activation_token_expires_at, self.ahora + timedelta(hours=48)
        )
        self.repositorio.find_by_email.assert_called_once_with("ana.perez@example.com")
        self.repositorio.save.assert_called_once()
        self.remitente.send_activation.assert_called_once_with(
            "ana.perez@example.com", resultado.activation_token
        )

    def test_registro_email_duplicado_no_guarda_ni_envia(self):
        existente = Usuario(
            email="ana@test.local", password_hash="hash", tipo=Usuario.TIPO_EMPRESA
        )
        self.repositorio.find_by_email.return_value = existente

        with self.assertRaisesRegex(ValueError, "ya está registrado"):
            self.service.register_user(
                "ana@test.local", "Abcdefg1", "Abcdefg1", Usuario.TIPO_EMPRESA
            )

        self.repositorio.save.assert_not_called()
        self.remitente.send_activation.assert_not_called()
```

- **Resultados y reporte de cobertura de código**

Coverage report: 98%

Files | Functions | Classes

coverage.py v7.16.1, created at 2026-11-03 17:01 -0500

File	function	statements	missing	excluded	coverage
src/main/application/usuario_application_service.py	UsuarioApplicationService.__init__	3	0	0	100%
src/main/application/usuario_application_service.py	UsuarioApplicationService.register_user	17	0	0	100%
src/main/application/usuario_application_service.py	UsuarioApplicationService.authenticate	5	0	0	100%
src/main/application/usuario_application_service.py	UsuarioApplicationService.recover_password	9	0	0	100%
src/main/application/usuario_application_service.py	UsuarioApplicationService.resend_activation	9	0	0	100%
src/main/application/usuario_application_service.py	UsuarioApplicationService.activate_account	10	1	0	90%
src/main/application/usuario_application_service.py	UsuarioApplicationService.reset_password	11	1	0	91%
src/main/application/usuario_application_service.py	UsuarioApplicationService.find_by_id	2	0	0	100%
src/main/application/usuario_application_service.py	UsuarioApplicationService._require_repository	2	0	0	100%
src/main/application/usuario_application_service.py	UsuarioApplicationService._require_email_sender	2	0	0	100%
src/main/application/usuario_application_service.py	UsuarioApplicationService._normalize_email	6	0	0	100%
src/main/application/usuario_application_service.py	*(no function)*	17	0	0	100%
Total		**93**	**2**	**0**	**98%**

coverage.py v7.16.1, created at 2026-11-03 17:01 -0500